

Updated Reading Guide and Math Primer

This revision adds the missing pieces that make the full guide easier to learn, easier to revise, and stronger for interviews.

What this front matter does: it tells you how to read the document, what to memorize, and how to connect the symbols to plain English before you dive into the main chapters.

How to read this guide

- First pass: skim the intuition boxes, the worked examples, and the chapter summaries.
- Second pass: re-do the calculations yourself. Do not just read them.
- Third pass: practice explaining the model out loud in plain English.
- Memorize only the core formulas: $h(\theta) = \theta_0 + \theta_1 x$, $J(\theta) = (1 / 2m)$ sum of squared errors, and the gradient descent update rule.
- Keep a small symbol glossary beside you while reading.

One-page symbol glossary

Symbol	Plain-English meaning	What to remember
x	Input feature	The thing you know
y	Actual output	The real answer in the data
y-hat	Predicted output	What the model says
theta0	Intercept / starting value	Prediction when $x = 0$
theta1	Slope / weight	How much prediction changes when x increases by 1
h(theta)	Hypothesis / model prediction	The model's guess
J(theta)	Cost function	How wrong the model is
alpha	Learning rate	Step size in gradient descent

Theta explained in plain English

In machine learning, theta is not a constant by itself. It is a **parameter** the model learns from data. A parameter is a knob the model can turn while training.

For one-feature linear regression, the line you know from school is $y = mx + c$. The machine learning version is $h(\theta) = \theta_0 + \theta_1 x$. The meanings are the same: θ_0 is the starting value, and θ_1 is the steepness.

A simple way to think about it:

- θ_0 moves the line up or down.
- θ_1 tilts the line steeper or flatter.
- Training means finding the theta values that make the line fit the data best.

Common beginner mistake to avoid

Do not read theta as a magical symbol. Read it as "the learned setting of the model." That one sentence removes a lot of confusion.

LINEAR REGRESSION

The Complete Master Guide

From Absolute Zero to DeepMind

Mathematics • Intuition • Code • Interviews • Research

SECTION 1 — WHAT PROBLEM ARE WE TRYING TO SOLVE?

1.1 The Core Human Need: Prediction

Before we write a single equation, let us understand something fundamental: humans have always wanted to predict the future. A farmer wants to know: if I plant this much seed, how much grain will I harvest? A shop owner wants to know: if I spend this much on advertising, how many customers will walk in? A doctor wants to know: given a patient's age and weight, what is their blood pressure likely to be?

These are all prediction problems. Linear regression is one of the oldest and most powerful tools humans invented to answer such questions — using data from the past to make educated guesses about the future.

 *INTUITION: Think of linear regression as a smart calculator that learns from historical data and uses patterns in that data to make predictions.*

1.2 What is Supervised Learning?

Before we define supervised learning formally, let us think about how a child learns to recognise apples.

You show the child a red, round fruit and say: 'This is an apple.' You show another one: 'This is also an apple.' Then you show an orange and say: 'This is NOT an apple.' After many examples, the child can identify apples on their own.

This is exactly what supervised learning is. You provide a machine:

- Inputs (called features): the things you know. For example: the size of a house, the number of bedrooms.
- Outputs (called labels or targets): the thing you want to predict. For example: the price of the house.

The machine looks at many pairs of (input → output) and learns the relationship between them. Once trained, it can predict the output for new inputs it has never seen before.

 *NOTE: The word 'supervised' means a teacher is guiding the learning process by providing correct answers. In unsupervised learning, there are no correct answers — the machine finds patterns on its own. Linear regression is always supervised.*

1.3 Real-World Examples of Prediction

Let us look at concrete examples so you understand exactly why prediction matters:

Business Problem	Input (What We Know)	Output (What We Predict)
House pricing	Size (sq ft), bedrooms, location	Sale price (₹ or \$)
Sales forecasting	Ad spend, season, competitor price	Units sold next month
Revenue prediction	Website traffic, promotions, staff	Monthly revenue
Customer lifetime value	Age, purchase history, frequency	Total future spend
Manufacturing yield	Temperature, pressure, machine age	% of good products
Medical diagnosis	Age, weight, cholesterol level	Blood pressure
Salary estimation	Years of experience, education	Annual salary
Energy consumption	Outside temperature, building size	Electricity usage (kWh)

1.4 Why Was Linear Regression Invented?

Linear regression was invented in the early 1800s by a mathematician named Carl Friedrich Gauss. He needed to predict the orbit of the asteroid Ceres based on limited observations. He invented the method of 'least squares' — which is the mathematical engine behind linear regression.

Later, Francis Galton (around 1885) used it to study how the heights of children relate to the heights of their parents. He noticed something remarkable: tall parents tend to have tall children, but not as tall as themselves. Short parents tend to have short children, but not as short as themselves. The children's heights 'regressed' (moved back) toward the average height of the population. This is why the technique is called REGRESSION.

💡 *INTUITION: The name 'regression' comes from 'regressing toward the mean' — a statistical phenomenon where extreme values tend to be followed by values closer to the average.*

1.5 The Promise of Linear Regression

Linear regression makes a specific assumption about the world: that there is a straight-line relationship between your inputs and your output. This sounds simplistic, but it is surprisingly

powerful. Many real-world relationships are approximately linear, especially when we look at small ranges of values.

Even when relationships are not perfectly linear, linear regression gives us a useful first approximation — a baseline that more complex models try to improve upon.

SECTION 2 — MATHEMATICAL FOUNDATIONS (From Absolute Zero)

2.1 Variables

A variable is a placeholder — a container that holds a value that can change. Just like a box labelled 'price' can hold ₹5000 today and ₹5500 tomorrow, a variable can hold different values at different times.

In mathematics, we use single letters to represent variables. The most common ones are x and y , but any letter works: a , b , z , p , q , etc.

Example:

- x = the number of years of experience a person has
- y = the person's annual salary in ₹ lakhs
- If $x = 3$, that means 3 years of experience
- If $y = 8$, that means ₹8 lakhs per year salary

 **NOTE:** Variables are just labels for quantities we care about. They have no fixed value — their value depends on the specific situation.

2.2 Constants

Unlike variables (which change), a constant has a fixed value that never changes. For example, the number of days in a non-leap year is always 365. The number π (pi) is always approximately 3.14159.

In mathematics, we often represent constants with letters too — commonly a , b , c , or Greek letters like α (alpha), β (beta), θ (theta). The key difference: when we say ' a is a constant,' we mean a has a specific numerical value in a given problem, even if we haven't written it out yet.

2.3 Understanding Coordinates and Graphs

A graph is a picture that helps us understand relationships between two variables. To draw a graph, we need a coordinate system — a grid with two perpendicular lines (axes).

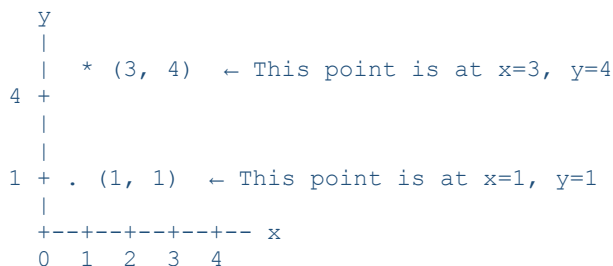


Figure 2.1 — A simple coordinate system with two points plotted

The horizontal line is called the x-axis. The vertical line is called the y-axis. They cross at a special point called the origin (0, 0).

Every point on a graph has two coordinates: an x-coordinate (how far right you go) and a y-coordinate (how far up you go). We write them as (x, y). So the point (3, 4) means: go 3 steps right, then 4 steps up.

2.4 Slope — The Rate of Change

Imagine you are hiking up a hill. The slope tells you: for every step you take forward, how many steps do you go up?

- A gentle hill: for every 10 steps forward, you go 1 step up. Slope = $1/10 = 0.1$ (gentle)
- A steep hill: for every 1 step forward, you go 2 steps up. Slope = $2/1 = 2$ (steep)
- Flat ground: for every step forward, you go 0 steps up. Slope = $0/\text{anything} = 0$ (flat)

In mathematics, slope is calculated as:

$$\text{Slope} = (\text{Change in } y) \div (\text{Change in } x)$$

We often write 'change in y' as Δy (the Greek letter delta Δ means 'change in') and similarly Δx for 'change in x'. So:

$$\text{Slope} = \Delta y / \Delta x = (y_2 - y_1) / (x_2 - x_1)$$

Worked Example — Calculating Slope:

Two points: Point A = (1, 2) and Point B = (4, 8)

Step 1: Find the change in y:

$$y_2 - y_1 = 8 - 2 = 6$$

Step 2: Find the change in x:

$$x_2 - x_1 = 4 - 1 = 3$$

Step 3: Divide:

$$\text{Slope} = 6 / 3 = 2$$

This means: for every 1 unit you move right on the x-axis, the line goes up by 2 units on the y-axis.

2.5 The Y-Intercept

The y-intercept is where a line crosses the y-axis (the vertical line). In other words, it is the value of y when x equals zero.

💡 *INTUITION: Think of the y-intercept as the 'starting value.' Before any input is applied ($x=0$), what is the output?*

Example: A taxi charges a flat starting fare of ₹50, then ₹15 per kilometre. If you travel 0 km, you still pay ₹50. This ₹50 is the y-intercept.

2.6 Functions — The Input-Output Machine

A function is a rule that takes an input and produces exactly one output. Think of it like a vending machine: you press a button (input), and you get a specific snack (output). Press the same button twice, you always get the same snack.

We write functions like this:

$$f(x) = 2x + 3$$

Here, f is the name of the function, x is the input, and ' $2x + 3$ ' is what we do with the input.

Let us calculate a few outputs:

Input x	Calculation	Output $f(x)$
0	$2(0) + 3 = 0 + 3$	3
1	$2(1) + 3 = 2 + 3$	5
2	$2(2) + 3 = 4 + 3$	7
5	$2(5) + 3 = 10 + 3$	13
-1	$2(-1) + 3 = -2 + 3$	1

2.7 Linear Equations — The Heart of Linear Regression

A linear equation is one where the highest power of x is 1 (just x , never x^2 or x^3). When you plot a linear equation, you always get a straight line.

The standard form is:

$$y = mx + c$$

Where:

- y = the output value (what we want to find)
- x = the input value (what we know)
- m = the slope of the line (how steeply it rises or falls)
- c = the y -intercept (where the line crosses the y -axis)

WORKED EXAMPLE: Salary vs. Experience

Suppose we know: Salary (y) = $3x + 4$ (where x = years of experience, y = salary in ₹ lakhs)

Years of Experience (x)	Calculation	Salary in ₹ Lakhs (y)
0	$3(0) + 4 = 0 + 4$	4 (starting salary)

Years of Experience (x)	Calculation	Salary in ₹ Lakhs (y)
1	$3(1) + 4 = 3 + 4$	7
2	$3(2) + 4 = 6 + 4$	10
5	$3(5) + 4 = 15 + 4$	19
10	$3(10) + 4 = 30 + 4$	34

This tells us: each additional year of experience adds ₹3 lakhs to salary (slope = 3), and a fresh graduate starts at ₹4 lakhs (intercept = 4).

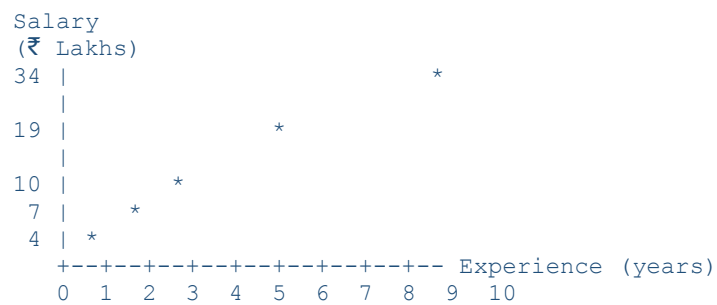


Figure 2.2 — Salary vs Experience plotted as a straight line

SECTION 3 — UNDERSTANDING THE LINE EQUATION

3.1 From School Maths to Machine Learning Notation

In school, you learned the equation of a straight line as:

$$y = mx + c$$

Machine learning uses a slightly different notation for the same idea:

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

At first glance, this looks completely different. But it is exactly the same equation! Let us decode every single symbol.

3.2 Decoding Every Symbol

What is h ?

h stands for hypothesis. In machine learning, we never claim our model is 'correct' — it is always an educated guess or hypothesis about the true relationship. The h reminds us that this is our model's best guess.

💡 *INTUITION: Why not just call it 'y'? Because y (the actual value from data) is something we observe. h is what our model predicts. They are often different! h is the prediction, y is the reality.*

What is the subscript θ in $h_{\theta}(x)$?

The θ (theta — a Greek letter) represents the parameters of our model. Writing θ as a subscript to h means: 'this hypothesis uses the specific values of θ that we have chosen.' Different values of θ give different lines. The subscript reminds us that h depends on θ .

What is x ?

x is the input feature — the thing we know and feed into the model. In the salary example, x would be years of experience. In the house price example, x would be the size in square feet.

What is θ_0 ?

θ_0 (theta-zero or theta-naught) is the bias or intercept. It plays the same role as c in $y = mx + c$.

Why 'bias'? Because even when $x = 0$ (no input), the prediction is still θ_0 — the model is 'biased' toward starting at a certain value rather than zero.

Why the subscript 0? In machine learning, we number things starting from 0 (a programming convention). θ_0 is the 'zeroth' parameter — it works on a constant input of 1 (we always have a silent $x_0 = 1$ multiplied by θ_0).

What if θ_0 changes?

- $\theta_0 = 0$: The line passes through the origin (0,0). No starting value.
- $\theta_0 = 5$: The line starts at $y=5$ when $x=0$. Shifts the entire line up by 5.
- $\theta_0 = -3$: The line starts at $y=-3$. Shifts the entire line down by 3.

What is θ_1 ?

θ_1 (theta-one) is the weight or coefficient for the input x . It plays the same role as m (slope) in $y = mx + c$.

θ_1 tells us: for every 1-unit increase in x , how much does our prediction change?

What if θ_1 changes?

- $\theta_1 = 0$: The line is flat — x has no effect on predictions.
- $\theta_1 = 2$: Every 1-unit increase in x increases prediction by 2.
- $\theta_1 = -1.5$: Every 1-unit increase in x decreases prediction by 1.5.
- $\theta_1 = 10$: Very steep line — small changes in x cause large prediction changes.

3.3 Symbol Mapping Table

School Notation	ML Notation	Meaning	Also Called
y	$h_{\theta}(x)$ or $\hat{y}$	The predicted output	Prediction, output, label
m	θ_1	Slope / how much x affects y	Weight, coefficient
c	θ_0	Y-intercept / starting value	Bias, intercept
x	x or x_1	Input variable	Feature, predictor, independent variable
The equation $y=mx+c$	$h_{\theta}(x)=\theta_0+\theta_1x$	The line equation	Hypothesis, model

3.4 Worked Example with ML Notation

Suppose our trained model has: $\theta_0 = 4$ and $\theta_1 = 3$

Our hypothesis is: $h_{\theta}(x) = 4 + 3x$

Predictions for different experience levels:

x (experience)	Calculation	$h_{\theta}(x)$ (predicted salary)
0	$4 + 3(0) = 4 + 0$	₹4 lakhs
2	$4 + 3(2) = 4 + 6$	₹10 lakhs
5	$4 + 3(5) = 4 + 15$	₹19 lakhs
7	$4 + 3(7) = 4 + 21$	₹25 lakhs

📌 **NOTE:** In machine learning, we also often write the prediction as $\hat{y}$ (y-hat). The hat symbol (^) over a variable always means 'the estimated or predicted version of y.' So $\hat{y}$ and $h_{\theta}(x)$ are the same thing.

SECTION 4 — MANUAL CALCULATIONS (Step by Step)

4.1 Toy Dataset 1: Temperature vs Ice Cream Sales

Temperature °C (x)	Ice Cream Units Sold (y)
20	100
25	150
30	200
35	250
40	300

Just from looking at the data, a pattern is clear: every 5°C rise in temperature leads to 50 more ice creams sold. Let us verify this using the line equation $y = mx + c$.

Step 1: Calculate the slope (m)

Take any two points. Let us use (20, 100) and (40, 300).

$$\begin{aligned}m &= (y_2 - y_1) / (x_2 - x_1) \\m &= (300 - 100) / (40 - 20) \\m &= 200 / 20 \\m &= 10\end{aligned}$$

So the slope is 10. Every 1°C increase leads to 10 more ice creams sold.

Step 2: Calculate the intercept (c)

We use the equation $y = mx + c$ and solve for c. Use any point, say (20, 100):

$$\begin{aligned}100 &= 10 \times 20 + c \\100 &= 200 + c \\c &= 100 - 200 \\c &= -100\end{aligned}$$

Step 3: Write the complete equation

$$y = 10x - 100$$

Step 4: Verify with another data point

Let us check if (25, 150) fits:

$$\begin{aligned}y &= 10(25) - 100 \\y &= 250 - 100 \\y &= 150 \quad \checkmark \quad \text{Correct!}\end{aligned}$$

Step 5: Make a prediction

What if the temperature is 45°C tomorrow? Predicted ice cream sales:

$$\begin{aligned}y &= 10(45) - 100 \\y &= 450 - 100 \\y &= 350 \text{ units}\end{aligned}$$

4.2 Toy Dataset 2: Study Hours vs Exam Score

Study Hours (x)	Exam Score (y)
1	40
2	50
3	65
4	70
5	80

This data is messier — not a perfect straight line. Let us try to find the best-fitting line.

 **NOTE:** In real-world data, points rarely fall exactly on a line. Linear regression finds the 'best possible straight line' through messy data — the one that minimises total error. This is the core problem we will solve in Section 6.

Using the Least Squares Formulas

When data is not perfectly linear, we use formulas derived from calculus (we will derive these properly in Section 8). For now, here are the formulas:

$$\begin{aligned}m &= \frac{[n \cdot \Sigma(xy) - \Sigma(x) \cdot \Sigma(y)]}{[n \cdot \Sigma(x^2) - (\Sigma x)^2]} \\c &= \frac{[\Sigma y - m \cdot \Sigma x]}{n}\end{aligned}$$

Where n = number of data points, Σ means 'sum of all values'.

Let us build a calculation table:

x	y	x ²	xy
1	40	1	40
2	50	4	100
3	65	9	195
4	70	16	280
5	80	25	400
Σx = 15	Σy = 305	Σx ² = 55	Σxy = 1015

n = 5 (five data points)

Step 1: Calculate slope m

$$\begin{aligned}m &= [5 \times 1015 - 15 \times 305] / [5 \times 55 - 15^2] \\&= [5075 - 4575] / [275 - 225] \\&= 500 / 50 \\&= 10\end{aligned}$$

Step 2: Calculate intercept c

$$\begin{aligned}c &= [305 - 10 \times 15] / 5 \\&= [305 - 150] / 5 \\&= 155 / 5 \\&= 31\end{aligned}$$

Step 3: Final equation

$$y = 10x + 31$$

Step 4: Verify and check errors

x	Actual y	Predicted $\hat{y} = 10x+31$	Error = y - $\hat{y}$
1	40	10(1)+31=41	-1
2	50	10(2)+31=51	-1
3	65	10(3)+31=61	+4
4	70	10(4)+31=71	-1

x	Actual y	Predicted $\hat{y} = 10x+31$	Error = $y - \hat{y}$
5	80	$10(5)+31=81$	-1

The errors are small — our line fits the data well but not perfectly. This is normal and expected. The goal of linear regression is to minimise these errors as much as possible.

SECTION 5 — INTUITION OF MODEL TRAINING

5.1 How Does a Machine 'Learn'?

The word 'learning' in machine learning sounds magical, but it is actually very mechanical. Let us walk through exactly what happens.

Imagine you are playing a game where you must guess the weight of a suitcase. Here are the rules:

1. You make a guess.
2. Someone tells you how far off your guess was (the error).
3. You use that error information to adjust your next guess.
4. You repeat steps 1-3 thousands of times until your guesses are very accurate.

This is EXACTLY what a machine does during training.

5.2 The Three Steps of Machine Learning

Step 1: Initialise Parameters (Make a First Guess)

The machine starts with some initial values for θ_0 and θ_1 . These are usually random numbers or zeros. At this point, the model is terrible — it makes wild predictions. But that is okay, because it will improve.

 *INTUITION: Think of this as a student on their first day who knows nothing but is about to start learning.*

Step 2: Measure Error (How Wrong Are We?)

With the current parameters, the model makes predictions for every training example. It then compares each prediction to the actual correct answer. The difference is called the error or residual.

For example:

- Actual salary: ₹12 lakhs
- Model predicts: ₹9 lakhs
- Error: ₹12 - ₹9 = ₹3 lakhs (we underestimated by ₹3 lakhs)

Step 3: Update Parameters (Improve the Guess)

Here is the clever part: using the errors, the machine calculates exactly how to adjust θ_0 and θ_1 to reduce those errors. It then makes small adjustments — not a big jump, but a small, controlled step in the right direction.

This process is called Gradient Descent (we will study it deeply in Sections 7 and 8).

5.3 An Analogy: The Blindfolded Mountain Hiker

Imagine you are blindfolded and standing somewhere on a mountain range. Your goal is to find the lowest valley. You cannot see anything, but you can feel the ground under your feet.

Here is your strategy:

5. Feel which direction the ground slopes downward.
6. Take one small step in that direction.
7. Feel again. Take another step downward.
8. Repeat until you cannot go any lower — you have found a valley (the minimum).

💡 *INTUITION: In linear regression, the 'mountain' is the error landscape. The 'valleys' are positions of minimum error. Gradient descent is the process of taking small steps toward the lowest valley.*

5.4 Why Not Just Jump to the Answer?

A natural question: why take thousands of small steps? Why not compute the perfect answer in one shot?

The answer is:

- For simple linear regression (one feature), we CAN compute the exact answer mathematically. This is called the Normal Equation.
- But for complex models with thousands of features (as in deep learning), the exact computation is impossibly expensive. Gradient descent is far more efficient in those cases.
- Understanding gradient descent here prepares you for neural networks and deep learning, where exact solutions do not exist.

📌 *NOTE: This is why we teach gradient descent even for simple linear regression. The technique scales to problems the exact method cannot handle.*

SECTION 6 — THE COST FUNCTION

6.1 Why Do We Need a Cost Function?

We need a way to measure how wrong our model is — a single number that summarises the total error across all training examples. This number is called the cost or loss.

Once we have this number, we have a clear goal: make it as small as possible. The cost function turns the vague goal of 'fit the data well' into the precise mathematical goal of 'minimise $J(\theta)$.'

6.2 The Symbol $J(\theta)$ Explained

$J(\theta)$ is the cost function. Let us decode each part:

- J : The letter J is traditionally used for cost functions in machine learning (it comes from the word 'objective function' in optimization). There is no deeper meaning — it is just a convention.
- θ (theta): These are the parameters of our model (θ_0 and θ_1). The cost J depends on which θ values we choose.
- $J(\theta)$: 'The cost when our parameters are θ .' Different values of θ give different costs.

💡 *INTUITION: J is like a report card for your model. A low J means the model fits the data well. A high J means the model is making large errors.*

6.3 Building the Cost Function Step by Step

Step 1: Compute the error for one data point

The error (also called residual) for one data point is:

$$\text{Error} = \text{Actual value} - \text{Predicted value}$$

$$\text{Error} = y - h_{\theta}(x)$$

Example: Actual = 80, Predicted = 75. Error = $80 - 75 = 5$.

Step 2: Why not just sum the errors?

Suppose we have these errors for 4 data points: +5, -3, +2, -4.

Sum = $5 + (-3) + 2 + (-4) = 0$.

The sum is ZERO, but our model is clearly not perfect! The positive and negative errors cancel each other out. This is a problem.

Step 3: Square each error

The solution: square each error. Squaring makes all values positive, and it also penalises large errors more heavily.

Squaring the errors: $5^2 = 25$, $(-3)^2 = 9$, $2^2 = 4$, $(-4)^2 = 16$.

Sum of squared errors = $25 + 9 + 4 + 16 = 54$. (No longer zero!)

 **NOTE:** Squaring the errors is not arbitrary. It has deep mathematical justification from probability theory (Maximum Likelihood Estimation under Gaussian noise assumptions). For now, know that it works well in practice.

Step 4: Average the squared errors

We divide by m (the number of training examples) to get the average, so the cost does not simply grow as we add more data.

Mean Squared Error (MSE) = (Sum of squared errors) / $m = 54 / 4 = 13.5$

Step 5: Divide by 2 (a convenience trick)

The standard MSE cost function divides by $2m$ instead of m :

$$J(\theta) = (1/2m) \times \sum_i [h_{\theta}(x_i) - y_i]^2$$

Why divide by 2? When we take the derivative later (in gradient descent), the 2 from squaring and the $\frac{1}{2}$ cancel out neatly, simplifying the math. It does not change which θ values minimise J — it just makes the derivative cleaner.

6.4 Every Symbol in the Cost Function

Symbol	Meaning	Example Value
$J(\theta)$	The total cost (how wrong our model is)	13.5
m	Number of training examples	100 rows in our dataset
i	Index — which data point we are looking at	$i=1$ means first row
x_i	Input feature of the i -th training example	x_i = years of experience for row i
y_i	Actual correct output for i -th example	y_i = actual salary for row i
$h_{\theta}(x_i)$	Model's prediction for i -th example	$h_{\theta}(x_i) = \theta_0 + \theta_1 x_i$

Symbol	Meaning	Example Value
Σ	Sum over all training examples (i=1 to m)	Add up all terms
$[h_{\theta}(x_i) - y_i]^2$	Squared error for one example	(predicted - actual) ²

6.5 Manual Cost Calculation Example

Dataset: 4 (experience, salary) pairs. $\theta_0 = 2$, $\theta_1 = 5$.

i	x_i	y_i (actual)	$h_{\theta}(x_i) = 2+5x$	Error = h-y	Error ²
1	1	8	$2+5(1)=7$	$7-8=-1$	$(-1)^2=1$
2	2	12	$2+5(2)=12$	$12-12=0$	$0^2=0$
3	3	20	$2+5(3)=17$	$17-20=-3$	$(-3)^2=9$
4	4	23	$2+5(4)=22$	$22-23=-1$	$(-1)^2=1$

Sum of squared errors = $1 + 0 + 9 + 1 = 11$

$J(\theta) = (1/2m) \times 11 = (1/8) \times 11 = 1.375$

Now try $\theta_0 = 3$, $\theta_1 = 5$ — the intercept changed by 1:

i	x_i	y_i (actual)	$h_{\theta}(x_i) = 3+5x$	Error = h-y	Error ²
1	1	8	$3+5(1)=8$	$8-8=0$	0
2	2	12	$3+5(2)=13$	$13-12=1$	1
3	3	20	$3+5(3)=18$	$18-20=-2$	4
4	4	23	$3+5(4)=23$	$23-23=0$	0

Sum = $0 + 1 + 4 + 0 = 5$

$J(\theta) = (1/8) \times 5 = 0.625$

The second set of parameters gives a lower cost! So $\theta_0 = 3$, $\theta_1 = 5$ is a better model for this data.

SECTION 7 — OPTIMIZATION: GRADIENT DESCENT

7.1 The Geometric Intuition

Imagine the cost function $J(\theta)$ as a landscape. If we have just one parameter θ_1 , the landscape is like a curve (like a bowl or a valley shape). If we have two parameters θ_0 and θ_1 , the landscape is a 3D surface.

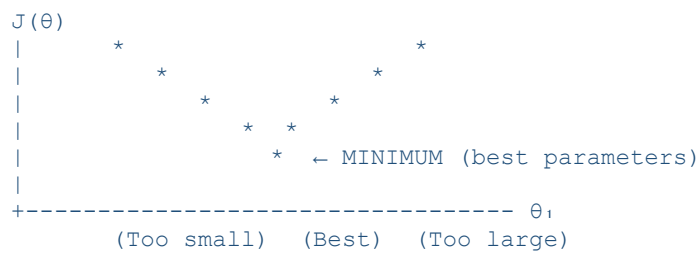


Figure 7.1 — The cost function J as a bowl-shaped curve. We want to find the bottom.

Our goal: find the value of θ that puts us at the bottom of this bowl (minimum J).

Gradient descent does this by:

- 9. Starting at a random position on the curve.
- 10. Measuring the slope at that position.
- 11. Taking a small step in the downhill direction.
- 12. Repeating until reaching the bottom.

7.2 What Is a Gradient?

In one dimension, the gradient is simply the slope (the derivative — how steep the curve is). In multiple dimensions, the gradient is a vector pointing in the direction of steepest ascent. We move in the OPPOSITE direction of the gradient to go downhill.

💡 *INTUITION: Imagine walking on a hillside. The gradient direction tells you which way is most uphill. To descend, you walk in the opposite direction.*

7.3 The Gradient Descent Update Rule

The update rule for gradient descent is:

$$\theta_j := \theta_j - \alpha \times \partial J(\theta) / \partial \theta_j$$

Let us decode every symbol:

Symbol	Meaning
θ_j	The j -th parameter (θ_0 or θ_1)

Symbol	Meaning
$:=$	Assignment: replace the left side with the right side's value
α (alpha)	Learning rate: how big each step is
$\partial J(\theta)/\partial \theta$	Partial derivative: the slope of J with respect to θ

7.4 The Learning Rate α

α (alpha) is a number you choose before training begins. It controls how large each step is.

- α too large: You take huge steps and might overshoot the minimum, bouncing around or even diverging (cost increases instead of decreasing)!
- α too small: You take tiny steps and eventually reach the minimum, but it takes an extremely long time.
- α just right: You converge smoothly and efficiently to the minimum.



Figure 7.2 — Effect of learning rate on convergence

7.5 The Computed Gradients for Linear Regression

After computing the partial derivatives (we derive these in Section 8), the gradient descent update rules for linear regression become:

$$\begin{aligned}\theta_0 &:= \theta_0 - \alpha \times (1/m) \times \sum_i [h_{\theta}(x_i) - y_i] \\ \theta_1 &:= \theta_1 - \alpha \times (1/m) \times \sum_i [h_{\theta}(x_i) - y_i] \times x_i\end{aligned}$$

Notice: θ_0 's update uses the errors alone. θ_1 's update multiplies each error by the corresponding x_i value. This makes sense: θ_1 controls how x affects the prediction, so it needs to know both the error AND the x value to update itself properly.

NOTE: CRITICAL: θ_0 and θ_1 must be updated **SIMULTANEOUSLY** in each iteration. Do **NOT** update θ_0 first, then use the new θ_0 to compute the gradient for θ_1 . Compute both gradients first, then update both at once.

7.6 Manual Gradient Descent — Worked Example

Dataset: $\{(1,2), (2,4), (3,6)\}$ — a perfect line $y = 2x$ with no intercept.

Start with: $\theta_0 = 0$, $\theta_1 = 0$. Learning rate $\alpha = 0.1$. $m = 3$.

Iteration 1:

Calculate predictions:

$$\begin{aligned}h(1) &= 0 + 0(1) = 0 & \text{Error} &= 0-2 = -2 \\h(2) &= 0 + 0(2) = 0 & \text{Error} &= 0-4 = -4 \\h(3) &= 0 + 0(3) = 0 & \text{Error} &= 0-6 = -6\end{aligned}$$

Calculate gradients:

$$\begin{aligned}\text{Gradient for } \theta_0 &= (1/3) \times [(-2) + (-4) + (-6)] = (1/3) \times (-12) = -4 \\ \text{Gradient for } \theta_1 &= (1/3) \times [(-2)(1) + (-4)(2) + (-6)(3)] \\ &= (1/3) \times [-2 + (-8) + (-18)] \\ &= (1/3) \times (-28) = -9.333\end{aligned}$$

Update parameters:

$$\begin{aligned}\theta_0 &:= 0 - 0.1 \times (-4) = 0 + 0.4 = 0.4 \\ \theta_1 &:= 0 - 0.1 \times (-9.333) = 0 + 0.933 = 0.933\end{aligned}$$

After 1 iteration: $\theta_0 = 0.4$, $\theta_1 = 0.933$. Still not perfect, but much better!

After many iterations, gradient descent will converge to approximately $\theta_0 \approx 0$ and $\theta_1 \approx 2$.

SECTION 8 — DERIVATIVES FROM ABSOLUTE ZERO

8.1 What is a Derivative?

A derivative measures how fast something is changing at a specific point. Let us build this up from scratch.

Suppose you are driving a car. Your position changes over time. At any given moment, your speedometer tells you how fast your position is changing — that is a derivative! Speed is the derivative of position with respect to time.

💡 *INTUITION: The derivative of $f(x)$ at a point tells you: if x increases by a tiny amount, how much does $f(x)$ change? It is the instantaneous rate of change.*

8.2 Slope as a Foundation

You already know how to calculate slope between two points: rise over run, or $\Delta y / \Delta x$. The derivative extends this idea to a single point.

If we have $f(x)$ and we want the slope at exactly one point, we take two very close points and shrink the distance between them to virtually zero:

$$f'(x) = \lim_{\Delta x \rightarrow 0} [f(x + \Delta x) - f(x)] / \Delta x$$

The symbol $f'(x)$ (f prime of x) means 'the derivative of f at x.' The phrase ' $\lim_{\Delta x \rightarrow 0}$ ' means 'as we make Δx smaller and smaller.'

8.3 Basic Derivative Rules (All You Need for Now)

Rule Name	If $f(x) =$	Then $f'(x) =$	Example
Constant Rule	c (any constant)	0	$f(x)=5, f'(x)=0$
Power Rule	x^n	$n \cdot x^{n-1}$	$f(x)=x^2, f'(x)=2x$
Constant Multiple	$c \cdot g(x)$	$c \cdot g'(x)$	$f(x)=3x^2, f'(x)=6x$
Sum Rule	$g(x) + h(x)$	$g'(x) + h'(x)$	$f(x)=x^2+x, f'(x)=2x+1$

8.4 Deriving the Gradient Descent Updates for Linear Regression

Now we will derive the gradient descent update rules from scratch, step by step.

Our cost function is:

$$J(\theta_0, \theta_1) = (1/2m) \times \sum_i [h_{\theta}(x_i) - y_i]^2$$

And $h_{\theta}(x) = \theta_0 + \theta_1 x$.

Deriving the gradient with respect to θ_0

We need $\partial J / \partial \theta_0$ — the derivative of J treating θ_1 as a constant.

$$\begin{aligned} J &= (1/2m) \times \sum [h_{\theta}(x_i) - y_i]^2 \\ &= (1/2m) \times \sum [\theta_0 + \theta_1 x_i - y_i]^2 \end{aligned}$$

Let $e_i = \theta_0 + \theta_1 x_i - y_i$ (the error for example i). Then $J = (1/2m) \sum e_i^2$

By the chain rule: $d(e_i^2)/d\theta_0 = 2e_i \times d(e_i)/d\theta_0$

And $d(e_i)/d\theta_0 = d(\theta_0 + \theta_1 x_i - y_i)/d\theta_0 = 1$ (since only θ_0 changes)

Therefore:

$$\partial J / \partial \theta_0 = (1/2m) \times \sum 2e_i \times 1 = (1/m) \times \sum [h_{\theta}(x_i) - y_i]$$

Deriving the gradient with respect to θ_1

Same process, but now $d(e_i)/d\theta_1 = x_i$ (since θ_1 is multiplied by x_i):

$$\partial J / \partial \theta_1 = (1/m) \times \sum [h_{\theta}(x_i) - y_i] \times x_i$$

These are the exact update gradients we used in Section 7!

💡 *INTUITION: The 2 from squaring and the 1/2 in the cost function cancelled out perfectly — that is exactly why we included the 1/2 in the cost function definition.*

SECTION 9 — LINEAR ALGEBRA FOR LINEAR REGRESSION

9.1 What is a Vector?

A vector is simply a list of numbers arranged in order. Think of it as a single column from a spreadsheet:

```
x = [1, 2, 3, 4, 5] (a row vector)
x = [1]              (a column vector)
    [2]
    [3]
```

Vectors are used to represent data efficiently. Instead of listing 100 salaries one by one, we put them all in a single vector y .

9.2 What is a Matrix?

A matrix is a rectangular grid of numbers — like a table. It has rows and columns. A matrix with m rows and n columns is called an $m \times n$ matrix.

```
X = [1  2] ← This is a 3x2 matrix (3 rows, 2 columns)
    [3  4]
    [5  6]
```

9.3 Why Machine Learning Uses Matrices

Suppose you have 1000 houses in your dataset, each with 3 features (size, bedrooms, age). Instead of writing 3000 separate numbers, you create one matrix X of shape (1000×3) . Every operation — predictions, gradients, updates — can then be expressed as a single matrix operation.

💡 *INTUITION: Matrix operations can be massively parallelised on modern hardware (GPUs). This is why deep learning is possible — without matrix math, training a neural network on millions of examples would take years instead of hours.*

9.4 Matrix Multiplication

Matrix multiplication is fundamental. To multiply matrix A ($m \times n$) by matrix B ($n \times p$), the result is a matrix C ($m \times p$). The rule:

```
C[i,j] = sum of A[i,k] × B[k,j] for all k
```

Simple example — 2x2 matrices:

$$\begin{aligned} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} &= \begin{bmatrix} (1 \times 5 + 2 \times 7) & (1 \times 6 + 2 \times 8) \\ (3 \times 5 + 4 \times 7) & (3 \times 6 + 4 \times 8) \end{bmatrix} \\ &= \begin{bmatrix} (5+14) & (6+16) \\ (15+28) & (18+32) \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix} \end{aligned}$$

9.5 The Normal Equation — Solving Linear Regression in One Shot

With matrix notation, linear regression has a beautiful closed-form solution called the Normal Equation:

$$\theta = (X^T X)^{-1} X^T y$$

Each symbol:

- θ : The vector of all parameters [$\theta_0, \theta_1, \theta_2, \dots$]
- X : The design matrix — all your input data ($m \times n+1$), with a column of 1s prepended for θ_0
- X^T : X transpose — flip rows and columns
- $(X^T X)^{-1}$: Matrix inverse of $X^T X$
- y : The vector of all target values

This formula gives you the optimal θ in a single computation — no iteration needed! But it requires computing a matrix inverse, which is expensive ($O(n^3)$) when you have many features.

Method	When to Use	Advantage	Disadvantage
Normal Equation	n features < ~10,000	Exact solution, no learning rate	Very slow for large n
Gradient Descent	Any number of features	Scales to millions of features	Needs learning rate, iterations

SECTION 10 — MULTIVARIATE LINEAR REGRESSION

10.1 From One Feature to Many

So far we have used a single input feature x to predict an output y . But real-world problems have multiple features. House prices depend on: size, number of bedrooms, number of bathrooms, location, age, etc.

The extension is straightforward. With n features, our hypothesis becomes:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \dots + \theta_n x_n$$

New symbols:

- $x_1, x_2, x_3, \dots, x_n$ — the n different input features
- $\theta_1, \theta_2, \dots, \theta_n$ — a separate weight for each feature
- θ_0 — still the bias/intercept

10.2 House Price Example with Multiple Features

Feature	Symbol	Meaning	Example Value
Size (sq ft)	x_1	Total floor area	1500
Bedrooms	x_2	Number of bedrooms	3
Age (years)	x_3	Age of the house	10
Distance to city (km)	x_4	Proximity to centre	5

Suppose our trained model has:

$$h_{\theta}(x) = 50000 + 300x_1 + 10000x_2 - 1000x_3 - 5000x_4$$

Let us decode this:

- $\theta_0 = 50,000$: Base price — a house with no area, no bedrooms, age 0, at city centre would cost ₹50,000 (this is the theoretical baseline).
- $\theta_1 = 300$: Each additional square foot adds ₹300 to the price.
- $\theta_2 = 10,000$: Each additional bedroom adds ₹10,000 to the price.
- $\theta_3 = -1,000$: Each additional year of age reduces price by ₹1,000.
- $\theta_4 = -5,000$: Each additional km from the city reduces price by ₹5,000.

Prediction for a specific house:

Size=1500, Bedrooms=3, Age=10, Distance=5

$$\begin{aligned}h_{\theta}(x) &= 50000 + 300(1500) + 10000(3) - 1000(10) - 5000(5) \\&= 50000 + 450000 + 30000 - 10000 - 25000 \\&= 495000\end{aligned}$$

Predicted price: ₹4,95,000.

10.3 Feature Scaling — Critical for Gradient Descent

When features have very different scales (e.g., size is in thousands while bedrooms is 2-5), gradient descent converges slowly and inefficiently.

Solution: scale all features to a similar range. Two common methods:

Method 1: Min-Max Normalisation

$$x_{\text{scaled}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$$

This scales every feature to the range [0, 1].

Method 2: Z-score Standardisation (Most Common)

$$x_{\text{scaled}} = (x - \mu) / \sigma$$

Where μ (mu) is the mean and σ (sigma) is the standard deviation. This centres the data around 0 with unit variance.

SECTION 11 — STATISTICAL FOUNDATIONS

11.1 Mean (Average)

The mean is the sum of all values divided by the count. It represents the 'centre' of a dataset.

$$\mu = (x_1 + x_2 + \dots + x_n) / n = (1/n) \sum x_i$$

Example: Salaries = [5, 8, 10, 12, 15] lakhs.

$$\text{Mean} = (5 + 8 + 10 + 12 + 15) / 5 = 50 / 5 = 10 \text{ lakhs}$$

11.2 Median

The median is the middle value when data is sorted. It is less affected by extreme values (outliers) than the mean.

Example: [5, 8, 10, 12, 15] — the median is 10 (third of five values).

If there is an even number of values, take the average of the two middle ones.

💡 *INTUITION: If a CEO earning ₹100 crore joins a team of 10 people earning ₹10 lakhs, the mean salary skyrockets but the median barely changes. This is why salary statistics often use median.*

11.3 Variance and Standard Deviation

Variance measures how spread out the data is from the mean.

$$\sigma^2 = (1/n) \times \sum (x_i - \mu)^2$$

Standard deviation (σ) is simply the square root of variance. It is easier to interpret because it is in the same units as the original data.

Example with salaries [5, 8, 10, 12, 15], mean = 10:

Deviations: (5-10)=-5, (8-10)=-2, (10-10)=0, (12-10)=2, (15-10)=5

Squared: 25, 4, 0, 4, 25

Variance $\sigma^2 = (25+4+0+4+25)/5 = 58/5 = 11.6$

Std dev $\sigma = \sqrt{11.6} \approx 3.41 \text{ lakhs}$

11.4 Covariance

Covariance measures how two variables change together.

$$\text{Cov}(x, y) = (1/n) \times \sum (x_i - \mu_x)(y_i - \mu_y)$$

- Positive covariance: when x is above its mean, y tends to be above its mean too (they move together).

- Negative covariance: when x is above its mean, y tends to be below its mean (they move oppositely).
- Near zero: x and y are relatively independent.

11.5 Correlation

Correlation standardises covariance to always fall between -1 and +1, making it easier to interpret:

$$r = \text{Cov}(x, y) / (\sigma_x \times \sigma_y)$$

Correlation Value	Meaning
$r = 1.0$	Perfect positive linear relationship
$r = 0.7$ to 0.9	Strong positive relationship
$r = 0.4$ to 0.7	Moderate positive relationship
$r \approx 0$	No linear relationship
$r = -0.4$ to -0.7	Moderate negative relationship
$r = -1.0$	Perfect negative linear relationship

 **NOTE:** Correlation measures linear relationships only. Two variables can have $r=0$ yet be strongly related in a non-linear way (e.g., a U-shaped relationship).

SECTION 12 — ASSUMPTIONS OF LINEAR REGRESSION

Linear regression is a powerful tool, but it rests on five key assumptions. Understanding these assumptions is crucial — not to memorise them, but to know when your model might be unreliable.

12.1 Linearity

What it means:

There must be a linear relationship between each feature x and the target y . The change in y per unit change in x must be constant.

How to detect violations:

Plot residuals (prediction errors) against predicted values. If you see a curved pattern instead of random scatter, linearity is violated.

What happens if violated:

Your model systematically over-predicts in some regions and under-predicts in others.

How to fix:

- Apply transformations to x (e.g., $\log(x)$, x^2)
- Use polynomial regression
- Switch to a non-linear model

12.2 Independence of Errors

What it means:

The errors for different data points should not be related to each other. Knowing one error should tell you nothing about another error.

Typical violation:

Time series data — yesterday's error predicts today's error (autocorrelation).

How to fix:

For time series, use models designed for temporal data (ARIMA, LSTMs).

12.3 Homoscedasticity (Equal Variance)

What it means:

The variance of errors should be constant across all levels of x . ('Homo' = same, 'scedastic' = variance.) The opposite is heteroscedasticity.

How to detect:

Plot residuals vs fitted values. If the spread of residuals increases (or decreases) as predicted values increase, you have heteroscedasticity.

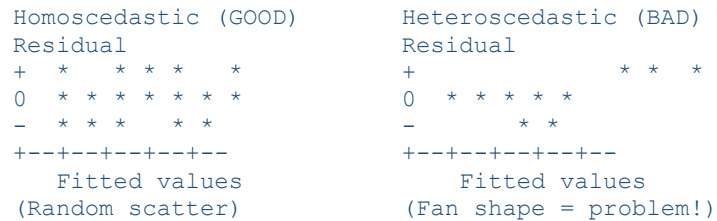


Figure 12.1 — Residual plots showing homoscedastic vs heteroscedastic errors

How to fix:

- Take $\log(y)$ as the target
- Use Weighted Least Squares

12.4 Normality of Errors

What it means:

The errors should follow a normal (bell-curve) distribution, centred at zero. This assumption matters primarily for hypothesis testing and confidence intervals, not for predictions.

How to detect:

Use a QQ plot (Quantile-Quantile plot) — if points fall near a diagonal line, errors are normally distributed.

How to fix:

Transform y (e.g., log transformation), or use a different model if errors are fundamentally non-normal.

12.5 No Multicollinearity

What it means:

In multivariate regression, the input features should not be highly correlated with each other. If two features are almost identical, the model cannot determine which one is responsible for changes in y .

Example: if you include both 'size in sq ft' and 'size in sq m' as features, they are perfectly correlated (one is exactly 10.764 times the other). The model will give unstable coefficient estimates.

How to detect:

Compute the Variance Inflation Factor (VIF) for each feature. $VIF > 10$ suggests severe multicollinearity.

How to fix:

- Remove one of the correlated features
- Combine them (e.g., via Principal Component Analysis)
- Use Ridge regression (see Section 18)

SECTION 13 — INTERPRETING COEFFICIENTS FOR BUSINESS STAKEHOLDERS

13.1 The Language of Coefficients

Once your model is trained, the coefficients (θ values) tell a story about your data. Being able to explain this story in plain English is a critical skill for data scientists.

Template for interpretation:

 **INTUITION:** "Holding all other features constant, a one-unit increase in [feature] is associated with a [coefficient value] [unit] change in [target]."

13.2 House Price Model — Full Interpretation

Trained model: $\text{Price} = 50,000 + 300 \times \text{Size} + 10,000 \times \text{Bedrooms} - 1,000 \times \text{Age} - 5,000 \times \text{Distance}$

Coefficient	Value	Business Interpretation
θ_0 (intercept)	₹50,000	The theoretical baseline price (not directly meaningful)
θ_1 (size)	+₹300	Every additional sq ft adds ₹300 to value (all else equal)
θ_2 (bedrooms)	+₹10,000	Each additional bedroom adds ₹10,000 (all else equal)
θ_3 (age)	-₹1,000	Each additional year of age reduces value by ₹1,000
θ_4 (distance)	-₹5,000	Each km further from city reduces value by ₹5,000

13.3 Pitfalls in Interpretation

 **Correlation is not causation!** Even if 'bedrooms' has a large positive coefficient, this does not mean adding a bedroom **CAUSES** the price to rise. There may be confounding variables.

Other pitfalls:

- Do not interpret the intercept literally in all cases — 'a house with 0 sq ft' is nonsensical.
- Coefficients change when you add or remove features — never interpret one in isolation from the others.

- Scaled features: if you standardised your features, coefficients are in 'standard deviation units,' not original units.

13.4 Feature Importance via Standardised Coefficients

To compare the relative importance of features, standardise all features first (z-score). Then the magnitude of each coefficient tells you its relative importance.

A feature with a standardised coefficient of 0.8 has more impact on the prediction than one with 0.3.

SECTION 14 — INTERVIEW PREPARATION (Beginner to MAANG)

14.1 Beginner Level Questions

Q: What is linear regression?

A: Linear regression is a supervised learning algorithm that models the linear relationship between one or more input features and a continuous output variable. It finds the best-fitting straight line through training data by minimising the sum of squared prediction errors.

Q: What is the difference between y and $\hat{y}$?

A: y is the actual observed value from the dataset. $\hat{y}$ (y-hat) is the value predicted by our model. The difference $y - \hat{y}$ is called the residual or error.

Q: What is the cost function in linear regression?

A: The cost function $J(\theta)$ measures how wrong the model's predictions are. We use Mean Squared Error: $J(\theta) = (1/2m) \times \sum (h_{\theta}(x_i) - y_i)^2$. It sums the squared differences between predictions and actual values. We minimise this to train the model.

Q: What is gradient descent?

A: Gradient descent is an iterative optimization algorithm. Starting from random parameter values, it repeatedly adjusts θ_0 and θ_1 in the direction that reduces the cost function, until it converges to the minimum. Think of it as walking downhill on the error surface.

14.2 Intermediate Level Questions

Q: What are the assumptions of linear regression?

A: Five main assumptions: (1) Linearity — linear relationship between features and target; (2) Independence — errors are independent of each other; (3) Homoscedasticity — constant error variance; (4) Normality — errors follow a normal distribution; (5) No multicollinearity — features are not highly correlated with each other.

Q: What is R-squared and what does it tell you?

A: R^2 (coefficient of determination) measures the proportion of variance in y that is explained by our model. $R^2 = 1 - (SS_{res}/SS_{tot})$, where SS_{res} is sum of squared residuals and SS_{tot} is total sum of squares. $R^2 = 0.85$ means our model explains 85% of the variance in y . It

ranges from 0 to 1 (occasionally negative for very poor models). Higher is better, but a high R^2 does not guarantee a good model.

Q: What is the difference between R^2 and Adjusted R^2 ?

A: R^2 always increases when you add more features, even useless ones. Adjusted R^2 penalises for adding features that don't improve the model: $\text{Adj } R^2 = 1 - [(1-R^2)(n-1)/(n-p-1)]$ where p is the number of features. Always use Adjusted R^2 when comparing models with different numbers of features.

Q: What is the difference between Ridge, Lasso, and Elastic Net?

A: All three add a regularization term to the cost function to reduce overfitting. Ridge (L2) adds $\alpha \times \sum \theta_i^2$ — shrinks all coefficients toward zero but keeps all features. Lasso (L1) adds $\alpha \times \sum |\theta_i|$ — can shrink coefficients to exactly zero, performing feature selection. Elastic Net combines both penalties, good when you have many features with some correlated ones.

14.3 Advanced / MAANG Level Questions

Q: What is the bias-variance tradeoff?

A: Any model's expected test error decomposes into: $\text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$. Bias is the error from oversimplified assumptions (underfitting). Variance is the error from excessive sensitivity to training data fluctuations (overfitting). A simple model (e.g., a flat line) has high bias/low variance. A complex model (e.g., high-degree polynomial) has low bias/high variance. The optimal model balances both. Regularisation increases bias but decreases variance, potentially reducing total error.

Q: Derive the Normal Equation from first principles.

A: We want to minimise $J(\theta) = (1/2) \|X\theta - y\|^2$. In matrix notation: $J = (1/2)(X\theta - y)^T(X\theta - y)$. Expanding: $J = (1/2)(\theta^T X^T X \theta - 2\theta^T X^T y + y^T y)$. Taking gradient: $\partial J / \partial \theta = X^T X \theta - X^T y = 0$. Solving: $X^T X \theta = X^T y$. If $X^T X$ is invertible: $\theta = (X^T X)^{-1} X^T y$.

Q: When would you use gradient descent over the Normal Equation?

A: Use gradient descent when the number of features n is large ($>10,000$). The Normal Equation requires computing $(X^T X)^{-1}$ which is $O(n^3)$ — prohibitively expensive for large n . Gradient descent is $O(kn)$ per iteration where k is the number of iterations, and scales much better. Also, gradient descent generalises to non-linear models (e.g., neural networks) while the Normal Equation does not.

Q: What happens to linear regression when features are perfectly correlated (multicollinearity)?

A: When features are perfectly correlated, $X^T X$ becomes singular (non-invertible). The Normal Equation fails completely. Gradient descent still works but the coefficient estimates become unstable — tiny changes in the data cause wild swings in coefficients. Ridge regression directly addresses this by adding αI to $X^T X$ before inverting: $\theta = (X^T X + \alpha I)^{-1} X^T y$, which is always invertible for $\alpha > 0$.

Q: How does linear regression relate to Maximum Likelihood Estimation?

A: If we assume the residuals follow a Gaussian (normal) distribution: $y_i = \theta^T x_i + \epsilon_i$ where $\epsilon_i \sim N(0, \sigma^2)$, then maximising the likelihood of the data is mathematically equivalent to minimising the sum of squared residuals. This is why MSE is the 'correct' loss for linear regression under Gaussian noise assumptions — it has deep probabilistic justification.

14.4 Research Level Questions

Q: How does linear regression connect to neural networks?

A: A neural network with no hidden layers, a linear activation function, and mean squared error loss IS linear regression. A single neuron with a sigmoid activation and binary cross-entropy loss is logistic regression. Neural networks are generalisations of linear models that learn complex non-linear representations through multiple layers of linear transformations followed by non-linear activations.

Q: What is the connection between linear regression and Gaussian Processes?

A: Bayesian linear regression with a Gaussian prior on θ leads to a posterior predictive distribution that is Gaussian. In the limit of infinite features with Gaussian priors, this extends to Gaussian Process regression — a non-parametric Bayesian method where any finite collection of predictions follows a joint Gaussian distribution. This is an active research area at organisations like DeepMind.

SECTION 15 — CODING FROM SCRATCH (No sklearn)

15.1 Complete Implementation

We will implement linear regression using only Python (and NumPy for array operations). Every line is explained.

```
import numpy as np
import matplotlib.pyplot as plt

# — Prediction Function —————
def predict(X, theta):
    '''
    X: input matrix of shape (m, n+1) — includes bias column
    theta: parameter vector of shape (n+1,)
    Returns: predictions of shape (m,)
    '''
    return X @ theta    # @ is matrix multiplication in Python

# Why X @ theta?
# For one example: h = theta0*1 + theta1*x1 + theta2*x2 ...
# In matrix form: h = X_row . theta (dot product)
# For ALL examples at once: H = X @ theta

# — Cost Function —————
def compute_cost(X, y, theta):
    '''
    Computes Mean Squared Error cost J(theta)
    J = (1/2m) * sum((h - y)^2)
    '''
    m = len(y)                # number of training examples
    predictions = predict(X, theta) # shape (m,)
    errors = predictions - y      # h - y for each example
    squared_errors = errors ** 2   # (h - y)^2 element-wise
    cost = (1 / (2 * m)) * np.sum(squared_errors)
    return cost

# — Gradient Descent —————
def gradient_descent(X, y, theta, alpha, num_iterations):
    '''
    X: design matrix (m, n+1)
    y: target vector (m,)
    theta: initial parameters (n+1,)
```

```

alpha: learning rate (float)
num_iterations: how many steps to take (int)
Returns: final theta, history of costs
'''
m = len(y)
cost_history = []

for iteration in range(num_iterations):
    # Step 1: Get predictions with current theta
    predictions = predict(X, theta)          # shape (m,)

    # Step 2: Compute errors
    errors = predictions - y                 # shape (m,)

    # Step 3: Compute gradient for ALL theta simultaneously
    # gradient = (1/m) * X^T @ errors
    # X^T shape: (n+1, m)
    # errors shape: (m,)
    # gradient shape: (n+1,)
    gradient = (1 / m) * (X.T @ errors)

    # Step 4: Update theta (simultaneous update!)
    theta = theta - alpha * gradient

    # Track cost for monitoring
    cost_history.append(compute_cost(X, y, theta))

return theta, cost_history

# — Training Example —————
# Generate synthetic salary data
np.random.seed(42)
X_raw = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]) # years experience
y = 3 * X_raw + 4 + np.random.randn(10) * 0.5      # true: y=3x+4+noise

# Add bias column (column of 1s for theta_0)
m = len(X_raw)
X = np.column_stack([np.ones(m), X_raw]) # shape (10, 2)

# Initialise parameters to zero
theta = np.zeros(2) # [theta_0, theta_1]

```

```

# Train
alpha = 0.01
num_iterations = 1000
theta_trained, costs = gradient_descent(X, y, theta, alpha,
num_iterations)

print(f'theta_0 (intercept): {theta_trained[0]:.4f}')
print(f'theta_1 (slope):      {theta_trained[1]:.4f}')
# Output should be close to theta_0≈4, theta_1≈3

# — Normal Equation (Alternative) —————
def normal_equation(X, y):
    '''Exact solution: theta = (X^T X)^{-1} X^T y'''
    return np.linalg.pinv(X.T @ X) @ X.T @ y # pinv = pseudo-inverse
    (safe)

theta_normal = normal_equation(X, y)
print(f'Normal Eq theta_0: {theta_normal[0]:.4f}')
print(f'Normal Eq theta_1: {theta_normal[1]:.4f}')

```

📌 **NOTE:** *`np.linalg.pinv()` is used instead of `np.linalg.inv()` because it handles near-singular matrices more gracefully — it is numerically stabler for real-world data.*

SECTION 16 — SCIKIT-LEARN IMPLEMENTATION

16.1 Full Pipeline with Explanations

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score,
mean_absolute_error

# — 1. Generate or Load Data —————
np.random.seed(0)
n = 200
experience = np.random.uniform(0, 15, n)
salary = 3.5 * experience + 5 + np.random.randn(n) * 2

df = pd.DataFrame({'experience': experience, 'salary': salary})
print(df.head())
print(df.describe())    # Always explore your data first!

# — 2. Split into Train and Test —————
# NEVER evaluate your model on data it trained on!
# test_size=0.2 means 20% for testing, 80% for training
X = df[['experience']] # Double brackets → DataFrame (not Series)
y = df['salary']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# — 3. Scale Features (best practice) —————
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # learn mean/std from
TRAIN only
X_test_scaled = scaler.transform(X_test)        # apply same transform
to TEST
# CRITICAL: Never fit the scaler on test data — that is data leakage!

# — 4. Train the Model —————
```

```

model = LinearRegression()
model.fit(X_train_scaled, y_train)

print(f'Intercept (theta_0): {model.intercept_:.4f}')
print(f'Coefficient (theta_1): {model.coef_[0]:.4f}')

# — 5. Make Predictions —————
y_pred_train = model.predict(X_train_scaled)
y_pred_test = model.predict(X_test_scaled)

# — 6. Evaluate —————
train_r2 = r2_score(y_train, y_pred_train)
test_r2 = r2_score(y_test, y_pred_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))
mae = mean_absolute_error(y_test, y_pred_test)

print(f'Train R2: {train_r2:.4f}')
print(f'Test R2: {test_r2:.4f}')
print(f'Test RMSE: {rmse:.4f}')
print(f'Test MAE: {mae:.4f}')

# — 7. Residual Plot —————
residuals = y_test - y_pred_test
plt.figure(figsize=(8, 5))
plt.scatter(y_pred_test, residuals, alpha=0.5, color='steelblue')
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel('Predicted Values')
plt.ylabel('Residuals')
plt.title('Residual Plot - Should Be Random Scatter Around Zero')
plt.tight_layout()
plt.savefig('residual_plot.png', dpi=150)

```

SECTION 17 — MODEL DIAGNOSTICS

17.1 R^2 — Coefficient of Determination

$$R^2 = 1 - SS_{res} / SS_{tot}$$

Where $SS_{res} = \sum (y_i - \hat{y}_i)^2$ (sum of squared residuals) and $SS_{tot} = \sum (y_i - \bar{y})^2$ (total sum of squares).

Interpretation: $R^2 = 0.85$ means our model explains 85% of the variation in y . The remaining 15% is unexplained.

R^2 Value	Interpretation	Typical Domain
0.9 - 1.0	Excellent fit	Physical sciences, engineering
0.7 - 0.9	Good fit	Many business applications
0.5 - 0.7	Moderate fit	Social sciences, economics
< 0.5	Weak fit — consider other models	Human behaviour prediction

17.2 Adjusted R^2

$$Adj\ R^2 = 1 - [(1 - R^2)(n - 1) / (n - p - 1)]$$

n = number of samples, p = number of features. Use when comparing models with different numbers of features.

17.3 RMSE — Root Mean Squared Error

$$RMSE = \sqrt{[(1/m) \times \sum (y_i - \hat{y}_i)^2]}$$

RMSE is in the same units as y , making it directly interpretable. An RMSE of ₹2 lakhs means our salary predictions are off by about ₹2 lakhs on average.

💡 *INTUITION: RMSE penalises large errors heavily due to squaring. Use it when large errors are especially undesirable.*

17.4 MAE — Mean Absolute Error

$$MAE = (1/m) \times \sum |y_i - \hat{y}_i|$$

MAE is also in the same units as y . It treats all error magnitudes equally (no squaring). More robust to outliers than RMSE.

17.5 MAPE — Mean Absolute Percentage Error

$$\text{MAPE} = (100/m) \times \sum |y_i - \hat{y}_i| / |y_i|$$

MAPE expresses error as a percentage of the actual value, making it scale-independent. Easy to communicate to business stakeholders: 'Our model is off by 8% on average.'

 *MAPE breaks down when actual values $y_i = 0$ or near zero (division by zero). Use SMAPE or MAE instead in such cases.*

17.6 Metric Selection Guide

Metric	Best When	Avoid When
R ²	Explaining model quality to stakeholders	Comparing models with different scales
Adj R ²	Comparing models with different features	Single-feature models (use R ² instead)
RMSE	Large errors are very costly	Outliers present in data
MAE	All errors treated equally	Large errors are particularly costly
MAPE	Need scale-independent % error	Target values can be zero

SECTION 18 — REGULARIZATION

18.1 The Overfitting Problem

Imagine a student who memorises every question in the textbook but cannot solve new problems. This is overfitting: the model learns the training data too perfectly, including its noise, and fails to generalise.

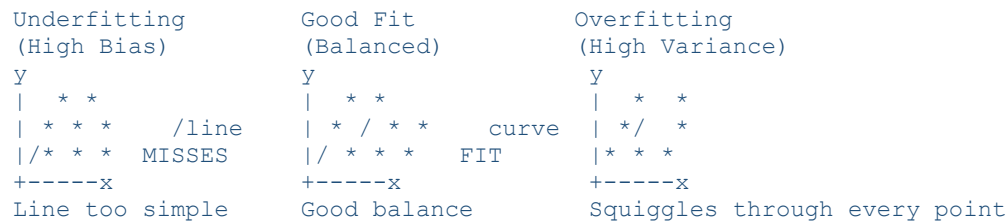


Figure 18.1 — Underfitting, good fit, and overfitting

18.2 Regularisation — Penalising Complexity

Regularisation adds a penalty term to the cost function that discourages large coefficient values. Large coefficients indicate a complex model that may be overfitting.

18.3 Ridge Regression (L2 Regularisation)

$$J_{\text{Ridge}}(\theta) = (1/2m) \times \sum (h_{\theta}(x_i) - y_i)^2 + \lambda \times \sum \theta_j^2$$

New symbol: λ (lambda) is the regularisation strength — a hyperparameter you choose. Larger λ = stronger penalty = simpler model.

- The penalty term $\lambda \times \sum \theta_j^2$ is the sum of squared coefficients times λ .
- This pushes all coefficients toward zero, but rarely exactly to zero.
- Note: θ_0 (the intercept) is typically NOT penalised.

Effect on the Normal Equation: $\theta = (X^T X + \lambda I)^{-1} X^T y$. The λI term makes the matrix invertible even with multicollinearity!

18.4 Lasso Regression (L1 Regularisation)

$$J_{\text{Lasso}}(\theta) = (1/2m) \times \sum (h_{\theta}(x_i) - y_i)^2 + \lambda \times \sum |\theta_j|$$

The penalty term uses absolute values $|\theta_j|$ instead of squares. This has a remarkable property: it can shrink some coefficients to exactly zero, effectively removing those features from the model. Lasso performs automatic feature selection!

Property	Ridge (L2)	Lasso (L1)
Penalty term	$\lambda \times \sum \theta_i^2$	$\lambda \times \sum \theta_i $
Effect on coefficients	Shrinks all toward zero	Some become exactly zero
Feature selection?	No — keeps all features	Yes — removes irrelevant features
When to use	All features relevant; multicollinearity	Many features; need sparse model
Computational difficulty	Has closed form	Requires iterative methods

18.5 Elastic Net

$$J_{\text{EN}}(\theta) = (1/2m) \times \sum (h - y)^2 + \lambda_1 \times \sum |\theta_i| + \lambda_2 \times \sum \theta_i^2$$

Elastic Net combines both L1 and L2 penalties. Best of both worlds: can zero out irrelevant features (like Lasso) while handling correlated features (like Ridge). Most practical choice when you are unsure.

18.6 Choosing λ (Hyperparameter Tuning)

λ is a hyperparameter — you choose it before training. Too small: no regularisation, still overfitting. Too large: underfitting.

Use k-fold cross-validation to find the optimal λ :

13. Try many values of λ (e.g., 0.001, 0.01, 0.1, 1, 10, 100).
14. For each λ , perform k-fold cross-validation.
15. Choose the λ that minimises validation set error.

```
from sklearn.linear_model import RidgeCV, LassoCV
ridge = RidgeCV(alphas=[0.01, 0.1, 1, 10, 100], cv=5)
ridge.fit(X_train, y_train)
print(f'Best lambda: {ridge.alpha_}')
```

SECTION 19 — WHEN NOT TO USE LINEAR REGRESSION

19.1 Model Selection Framework

Situation	Use Instead	Why
Binary output (yes/no, 0/1)	Logistic Regression	Linear regression can predict outside [0,1]
Multi-class classification	Softmax / SVM / Tree	Linear regression is for continuous output only
Highly non-linear relationships	Decision Tree, XGBoost	Better at capturing complex patterns
Very many correlated features	PCA + Linear, Ridge	Multicollinearity causes instability
Image/audio/text data	Neural Networks, CNNs, LLMs	Unstructured data needs representation learning
Time series with trends/seasonality	ARIMA, LSTM, Prophet	Captures temporal dependencies
Outliers dominate data	Robust Regression, Huber	MSE is very sensitive to outliers

19.2 Decision Tree for Model Selection

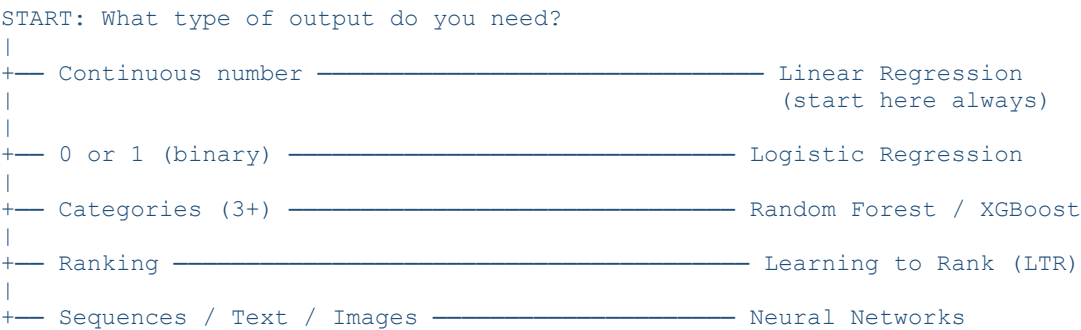


Figure 19.1 — Simple model selection guide

19.3 The 'Always Start with Linear Regression' Rule

💡 *INTUITION: ALWAYS start with linear regression. It is fast to train, easy to interpret, and provides a baseline. If it performs well, you are done. If not, you understand why a more complex model is needed — and by how much it needs to improve.*

This is not just practical advice — top ML teams at Google, Meta, and Airbnb follow this principle. Simple baselines prevent unnecessary complexity.

SECTION 20 — RESEARCH PERSPECTIVE: FROM LINEAR REGRESSION TO DEEPMIND

20.1 The Surprising Depth of Linear Regression

Linear regression is not a 'beginner topic' that researchers move past. It is a living research area with deep connections to every advanced field in machine learning.

20.2 Connections to Modern AI

Linear Regression Concept	Modern AI Connection
MSE loss function	Foundation for VAE reconstruction loss, neural network regression heads
Gradient descent	The backbone of backpropagation, Adam, RMSprop — all optimization in deep learning
Regularization (L1/L2)	Weight decay in neural networks; the basis of modern regularisation research
Normal Equation	Moore-Penrose pseudoinverse appears in attention mechanisms, linear algebra theory
Bias-variance tradeoff	Core of double descent phenomenon — active research in 2024/2025
Feature representation	Directly motivates representation learning and embeddings in transformers
Bayesian linear regression	Connects to Gaussian Processes, BNNs — used in DeepMind AlphaFold uncertainty

20.3 The Double Descent Phenomenon — Cutting Edge Research

Classical statistics says: more model complexity → overfitting after a point. But recent research (Belkin et al., 2019 — actively cited in DeepMind work) discovered that if you continue increasing model complexity FAR beyond the interpolation threshold, test error starts DECREASING again.

This double descent curve challenges the classical bias-variance tradeoff and is closely related to overparameterised linear regression. Understanding linear regression deeply is the entry point to this frontier research.

20.4 Linear Regression as Neural Network

A neural network with zero hidden layers and no activation function IS linear regression. The entire theoretical toolkit for linear regression — loss functions, optimisation, generalisation bounds — directly applies to understanding the behaviour of the final layers of deep networks. Researchers at DeepMind use linear probing (training a linear regression head on frozen neural network embeddings) to understand what representations networks learn.

SECTION 21 — DEEPMIND PREPARATION ROADMAP

21.1 The Full Learning Progression

Stage	Topics	Why It Matters	Timeline
1. Linear Regression	This entire document	Foundation for all of ML	Weeks 1-2
2. Classification	Logistic regression, SVM, decision trees	Supervised learning toolkit	Weeks 3-4
3. Statistics	Hypothesis testing, distributions, Bayes theorem	Rigorous reasoning under uncertainty	Weeks 5-6
4. Probability	Random variables, expectation, MLE, MAP	Probabilistic ML foundation	Weeks 7-8
5. Linear Algebra	Eigenvalues, SVD, PCA, matrix decomp.	Underpins all of deep learning	Weeks 9-10
6. Optimization	Convexity, SGD, Adam, Newton methods	Training algorithms beyond gradient descent	Weeks 11-12
7. Neural Networks	Backprop, CNNs, RNNs, attention	Deep learning toolkit	Months 4-6
8. Deep Learning	Transformers, diffusion, GNNs	Modern architectures	Months 7-9
9. Reinforcement Learning	MDPs, Q-learning, policy gradients	Core DeepMind research area	Months 10-14
10. Research Skills	Paper reading, experimentation, writing	Required for research roles	Ongoing

21.2 Recommended Resources by Stage

Linear Regression and ML Foundations

- Book: 'Hands-On Machine Learning' by Aurélien Géron (Chapters 1-4)
- Course: Andrew Ng's Machine Learning Specialisation (Coursera)
- Practice: Kaggle micro-courses — free and practical

Mathematics

- Linear Algebra: '3Blue1Brown — Essence of Linear Algebra' (YouTube series)
- Calculus: Khan Academy Multivariable Calculus — free, excellent animations
- Statistics: 'Statistics' by Freedman, Pisani, Purves — intuition-first

Deep Learning

- Book: 'Deep Learning' by Goodfellow, Bengio, Courville — the bible
- Course: Fast.ai — practical deep learning, top-down approach
- Course: DeepLearning.AI Deep Learning Specialisation (Coursera)

Reinforcement Learning (DeepMind Focus)

- Book: 'Reinforcement Learning: An Introduction' by Sutton and Barto — free PDF online
- Course: David Silver's RL lectures (UCL) — available on YouTube
- Paper: 'Human-level control through deep reinforcement learning' — DQN paper by DeepMind

21.3 Research Skills Development

16. Paper reading: Start with survey papers, then work backward to foundational papers.
17. Implementation: Reproduce key results from papers — this builds deep understanding.
18. Writing: Maintain a technical blog or GitHub repository explaining concepts.
19. Community: Follow ML researchers on Twitter/X, attend NeurIPS workshops online.

SECTION 22 — MAANG-LEVEL PROJECT: House Price Prediction Engine

22.1 Project Overview

We will design a production-grade regression project using the California Housing Dataset (publicly available via sklearn). This project demonstrates every skill an interviewer at Google, Meta, or a top AI research lab looks for.

Component	What We Build	Technology
Data ingestion	Load and validate raw data	Pandas, Great Expectations
EDA	Statistical and visual exploration	Matplotlib, Seaborn, Plotly
Feature engineering	Create meaningful features	Pandas, scikit-learn Pipelines
Modelling	Linear, Ridge, Lasso, Elastic Net	scikit-learn
Experiment tracking	Log all runs with parameters	MLflow
Evaluation	Multiple metrics + diagnostic plots	scikit-learn, matplotlib
Deployment	REST API endpoint	FastAPI
Monitoring	Log predictions, detect drift	Custom + Evidently AI
Documentation	Full README + Architecture diagram	Markdown, diagrams.net

22.2 Project Directory Structure

```
house_price_predictor/
├── README.md
├── requirements.txt
├── setup.py
├── .github/
│   └── workflows/
│       └── ci.yml                # Automated tests on push
├── data/
│   ├── raw/                     # Never modified
│   └── processed/               # Cleaned, feature-engineered
└── notebooks/
```



```

|   |─ 01_EDA.ipynb          # Exploratory Data Analysis
|   |─ 02_feature_eng.ipynb
|   └─ 03_modelling.ipynb
└─ src/
    |─ __init__.py
    |─ data/
    |   └─ preprocess.py    # Data cleaning pipeline
    |─ features/
    |   └─ engineer.py      # Feature creation
    |─ models/
    |   |─ train.py         # Training loop
    |   └─ evaluate.py     # Metrics computation
    └─ api/
        └─ app.py          # FastAPI server
└─ tests/
    |─ test_preprocessing.py
    └─ test_model.py
└─ mlruns/                  # MLflow tracking files
└─ models/                  # Saved model artifacts

```

22.3 Key Code — Training Pipeline

```

# src/models/train.py
import mlflow
import mlflow.sklearn
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_score

def train_model(X_train, y_train, alpha=1.0):
    with mlflow.start_run():
        # Log hyperparameters
        mlflow.log_param('model_type', 'Ridge')
        mlflow.log_param('alpha', alpha)

        # Build pipeline (scaler + model in one step)
        pipeline = Pipeline([
            ('scaler', StandardScaler()),
            ('model', Ridge(alpha=alpha))
        ])

```

```
# Train
pipeline.fit(X_train, y_train)

# Cross-validate
cv_scores = cross_val_score(
    pipeline, X_train, y_train, cv=5, scoring='r2'
)

# Log metrics
mlflow.log_metric('cv_r2_mean', cv_scores.mean())
mlflow.log_metric('cv_r2_std', cv_scores.std())

# Save model
mlflow.sklearn.log_model(pipeline, 'model')

return pipeline, cv_scores.mean()
```

SECTION 23 — GITHUB PORTFOLIO

23.1 README Template

```
# House Price Prediction Engine

[![CI Status](your_badge_url)]
[![Python 3.10+](badge_url)]
[![License: MIT](badge_url)]

## Problem
Predict California house prices using multiple linear regression
variants with full MLOps pipeline.

## Key Results
| Model           | Test R2 | RMSE      |
|-----|-----|-----|
| Baseline mean  | 0.00      | 115,342 |
| Linear Reg     | 0.64      | 68,912  |
| Ridge ( $\alpha=1$ ) | 0.67      | 65,100  |
| Lasso ( $\alpha=0.1$ ) | 0.66      | 66,210  |
| Elastic Net    | 0.68      | 64,890  |

## Architecture
[Raw Data] → [Preprocessing] → [Feature Eng]
→ [Model Training + MLflow] → [Evaluation]
→ [FastAPI Deployment] → [Evidently Monitoring]

## Quick Start
git clone https://github.com/yourusername/house-price-predictor
cd house-price-predictor
pip install -r requirements.txt
python src/models/train.py
uvicorn src.api.app:app --reload
```

23.2 Portfolio Checklist

Item	Purpose	Status
Comprehensive README with results table	First impression	Must have

Item	Purpose	Status
Jupyter notebooks with visualisations	Shows EDA skills	Must have
Clean, modular Python code	Shows engineering skills	Must have
Unit tests	Shows professionalism	Strongly recommended
MLflow experiment tracking	Shows MLOps awareness	Strongly recommended
FastAPI deployment	Shows end-to-end thinking	Recommended
GitHub Actions CI/CD	Shows DevOps awareness	Nice to have
Architecture diagram	Shows system design thinking	Nice to have

SECTION 24 — REVISION GUIDE

24.1 One-Day Revision Plan

Hour	Topic	Goal
Hour 1	Sections 1-3	Problem, foundations, line equation
Hour 2	Sections 5-6	Training intuition, cost function
Hour 3	Sections 7-8	Gradient descent, derivatives
Hour 4	Sections 12, 17-18	Assumptions, metrics, regularisation
Hour 5	Section 14	Answer 10 interview questions aloud
Hour 6	Section 15-16	Code from memory

24.2 Seven-Day Mastery Plan

Day	Focus	Activity
Day 1	Theory deep dive	Read Sections 1-9 carefully
Day 2	Maths practice	Redo all manual calculations from scratch on paper
Day 3	Coding	Implement linear regression from scratch without looking
Day 4	Assumptions and diagnostics	Read 12, 17. Draw residual plots by hand
Day 5	Regularisation	Implement Ridge and Lasso; tune lambda with CV
Day 6	Interview prep	Answer every question in Section 14 aloud; time yourself
Day 7	Project	Build and push the Section 22 project to GitHub

24.3 Thirty-Day Deep Mastery Plan

Week	Focus	Key Milestone
Week 1	Linear regression mastery	Score 90%+ on 50-question test
Week 2	Statistics and probability review	Understand MLE/MAP derivation
Week 3	Production project + MLflow	Published GitHub project with results table
Week 4	Interview circuit	5 mock interviews; advance to logistic regression

24.4 Quick Reference — Key Formulas

Formula	Name	Use
$h_{\theta}(x) = \theta_0 + \theta_1 x$	Hypothesis	Single-feature prediction
$J(\theta) = (1/2m)\sum(h-y)^2$	MSE Cost Function	Measure total error
$\theta_0 := \theta_0 - \alpha \times \partial J / \partial \theta_0$	Gradient Descent Update	Parameter update rule
$\theta = (X^T X)^{-1} X^T y$	Normal Equation	Exact solution (small n)
$R^2 = 1 - SS_{res} / SS_{tot}$	R-squared	Explained variance
$RMSE = \sqrt{MSE}$	Root MSE	Error in original units
$J_{Ridge} = J + \lambda \sum \theta_i^2$	Ridge Cost	L2 regularisation
$J_{Lasso} = J + \lambda \sum \theta_i $	Lasso Cost	L1 regularisation (sparse)

END OF DOCUMENT

Linear Regression: From Absolute Zero to DeepMind

Missing Chapters Added for Depth and Interview Readiness

The main guide already covers prediction, equations, cost, gradient descent, assumptions, diagnostics, regularization, interviews, and the DeepMind roadmap. These extra chapters fill the gaps that make the guide even stronger for real work.

Preprocessing before modeling

Why this matters:

- Split data into train, validation, and test sets before you train anything.
- Fit preprocessing steps only on the training set, then apply them to validation and test sets. This avoids data leakage.
- Scale numeric features when units are very different.
- Encode categorical variables with one-hot encoding.
- Use cross-validation when the dataset is small or when you want a more stable estimate of performance.

Outliers and robust alternatives

Practical rule:

- Linear regression with MSE can be strongly influenced by outliers.
- Use MAE when you want a loss that is less sensitive to extreme values.
- Use Huber loss when you want a compromise: quadratic for small errors, linear for large errors.
- Use robust regression when the data has obvious contamination or heavy-tailed noise.

Statistical inference

Interview depth:

- Prediction tells you what the model does; inference tells you what the coefficients mean and how uncertain they are.
- Confidence intervals show a range of plausible values for each coefficient.
- A p-value helps test whether a coefficient is likely different from zero, but it should not be treated as the only decision rule.
- For interviews, be ready to explain significance cautiously, especially when features are correlated.

Common mistakes checklist

Quick revision:

- Do not mix up y and $\hat{y}$.
- Do not assume the intercept always has a real-world meaning.
- Do not assume correlation means causation.
- Do not scale using the full dataset before splitting.

- Do not judge a model only by training performance.
- Do not interpret one coefficient in isolation when features are correlated.

How these additions fit the original guide

The main document already takes you from the problem statement to feature interpretation and model selection. These pages simply make the learning path cleaner: first understand the symbols, then the data pipeline, then the errors and uncertainty, and only then move to the broader project and interview story.